

VIETNAM NATIONAL UNIVERSITY, HANOI
INTERNATIONAL SCHOOL

—o0o—



REPORT OF FINAL PROJECT

Topic 8: Twitter Sentiment Classification Using Transformer Models
(BERT/RoBERTa): A Parameter-Efficiency and Cross-Model Error
Analysis Study

Subject:	Artificial Intelligence
Class:	INS 3080
Lecture:	Dr. Nguyen Viet Hung
Student's name	Student's ID
Ngo Thi Kim Ngan	23071029
Tong Khanh Ly	23071036
Nguyen Quang Minh	23071039
Nguyen Ngoc Anh	23071077

Twitter Sentiment Classification Using Transformer Models (BERT/RobERTa): A Parameter-Efficiency and Cross-Model Error Analysis Study

Ngo Thi Kim Ngan¹, Tong Khanh Ly¹, Nguyen Quang Minh¹, Nguyen Ngoc Anh¹

¹INTERNATIONAL SCHOOL, VIETNAM NATIONAL UNIVERSITY, HANOI, VIETNAM

Abstract

Sentiment classification on Twitter is complicated by short, informal, and noisy text containing hashtags, mentions, emojis, and non-standard grammar. We study this task on the TweetEval sentiment benchmark, comparing a TF-IDF + LinearSVC baseline against fine-tuned BERT and RoBERTa, and extend beyond standard fine-tuning along two directions largely unexplored on this benchmark: (i) a comparison of full fine-tuning against Low-Rank Adaptation (LoRA) across multiple ranks, evaluating the trade-off between parameter efficiency, training cost, and predictive performance, and (ii) a cross-model error analysis that isolates and categorises the tweets every architecture misclassifies. Our fine-tuned RoBERTa reaches 70.58% test accuracy, substantially outperforming the TF-IDF + LinearSVC baseline (56.68%) and fine-tuned BERT (67.60%). LoRA matches or marginally exceeds full fine-tuning at every rank tested, reaching 71.29% accuracy while updating as little as 0.59% of RoBERTa's parameters and training up to 39% faster, indicating that full-parameter updates are not strictly necessary to reach this benchmark's performance ceiling. The error analysis further shows that the 1,605 tweets misclassified by all three architectures stem predominantly from label ambiguity (42.0%) and missing context (33.7%) rather than the more commonly cited failure modes of sarcasm (0.2%) or slang (1.4%), offering a more precise account of where current Transformer-based approaches to Twitter sentiment classification remain limited.

Keywords: Sentiment Analysis, Twitter, BERT, RoBERTa, TweetEval, Low-Rank Adaptation (LoRA), Error Analysis

doi: International School, Vietnam National University – <https://vnu.edu.vn> -2026

1. Introduction

Sentiment classification refers broadly to the computational treatment and classification of opinion and sentiment in text, with common formulations including positive, neutral, and negative sentiment classes [1]. Tweets pose distinct challenges compared to the longer, well-formed text typically used to evaluate NLP systems: they are short, often ungrammatical, and rely heavily on platform-specific conventions such as hashtags, user mentions, emojis, and informal abbreviations. These properties make lexical, bag-of-words approaches unreliable, as sentiment on Twitter frequently depends on word order, negation scope, and context that such methods cannot capture.

To address this task, we use the TweetEval benchmark [2], a unified evaluation suite for Twitter-specific classification tasks, and its sentiment subset derived from SemEval-2017 Task 4 [3]. We first establish a TF-IDF + Linear SVM baseline, then fine-tune two Transformer-based language models, BERT

and RoBERTa, which encode each tweet through self-attention and are therefore better equipped to capture context, negation, and long-range dependencies than lexical baselines.

Beyond reproducing this standard fine-tuning pipeline, this project extends the analysis along two directions that have not been jointly examined on the TweetEval sentiment benchmark:

- **Parameter-efficient fine-tuning.** Fine-tuning a Transformer the standard way updates all of its parameters, which is computationally expensive. We compare full fine-tuning of RoBERTa against Low-Rank Adaptation (LoRA) [4] across four ranks ($r \in \{4, 8, 16, 32\}$), evaluating not only test accuracy but also the number of trainable parameters and training time, to determine whether a much smaller update budget can reach comparable performance on this task.
- **Cross-model error analysis.** Rather than reporting accuracy in isolation, we align the predictions of all three reproduced models (TF-IDF + SVM,

BERT, and RoBERTa) on the test set and isolate the tweets every architecture misclassifies at once, attributing each one to a linguistic cause (e.g. label ambiguity, missing context, negation, sarcasm) to identify which failure modes are systematic to the task itself rather than specific to any one model.

These two directions yield findings that run against common assumptions about this task: LoRA matches or marginally exceeds full fine-tuning while updating as little as 0.59% of RoBERTa’s parameters and training up to 39% faster, and the tweets every architecture misclassifies are driven predominantly by label ambiguity and missing context rather than by sarcasm, the failure mode most commonly cited in the literature. Together, these directions move beyond a single accuracy comparison to provide a more complete picture of the trade-offs - in computational cost and residual error patterns - involved in applying Transformer-based models to Twitter sentiment classification. The remainder of this report is organized as follows: Section 2 reviews related work, Section 3 describes the dataset, Section 4 details the preprocessing and model configurations (baseline, BERT/RoBERTa fine-tuning, LoRA, and the error-analysis framework), Section 5 reports experimental results, Section 6 discusses these results and their implications, and Section 7 concludes.

2. Literature Review

2.1. Twitter Sentiment Analysis Benchmarks

Twitter sentiment analysis is challenging because tweets are short, informal, and contain hashtags, mentions, abbreviations, and hyperlinks. SemEval-2017 Task 4 provides labeled tweets for positive, neutral, and negative sentiment classification [3]. TweetEval later introduced a unified benchmark for Twitter NLP tasks, including sentiment analysis [2]. This study uses TweetEval because its predefined training, validation, and testing sets support consistent model evaluation.

2.2. Transformer-based Language Models and Fine-tuning

Transformer models capture contextual relationships between words using self-attention [5]. BERT uses bidirectional context to better understand meaning [6], while RoBERTa has a more effective pre-training strategy [7]. Fine-tuning adapts a pre-trained model to a specific task using labeled data [6], allowing the model to reuse general language knowledge while learning patterns of different sentiment. In this study, BERT, RoBERTa, and Twitter-RoBERTa are fine-tuned on TweetEval and compared against the TF-IDF + LinearSVC baseline to evaluate the joint

benefit of contextual representations, Twitter-specific pre-training, and full task-adaptation.

2.3. Research Gap

LoRA has been shown to match full fine-tuning on RoBERTa, but only on well-formed, edited benchmarks [4]. It has not been tested on short, noisy, informal Twitter text, which is exactly what TweetEval was built to evaluate [2]. It is unclear whether LoRA’s efficiency advantage still holds on this harder kind of text.

Separately, a recent meta-review of 38 systematic reviews on sentiment analysis found sarcasm to be the most commonly cited challenge in the field [8]. But this consensus is rarely tested directly against the errors that a weak model, BERT, and RoBERTa all share, once each model’s own individual mistakes are filtered out.

Our contribution is to test both gaps on the same benchmark. First, we sweep LoRA across four ranks on TweetEval sentiment to see how much of RoBERTa’s full capacity is actually needed. Second, we isolate the tweets that every model gets wrong and identify what really causes them, to check whether sarcasm is really the main problem or not (Section 5.3). Together, these two experiments tell us not just how well the models perform, but where their remaining limitations actually come from.

3. Dataset

We use the TweetEval Sentiment dataset. The data set already has three splits: train, validation, and test. Each tweet can be assigned to one of the three categories: negative (0), neutral (1), and positive (2).¹

Data Loading Process . The dataset was loaded by using Pandas library. The text of the tweets and their associated sentiment labels were read from two separate files and were zipped together row-wise. Afterward, a few validation checks were performed to verify that each tweet had a unique corresponding label, prior to any further processing or analysis of the data.

Table 1. Dataset Size

Dataset	Tweets	Labels
Training	45,615	45,615
Validation	2,000	2,000
Testing	12,284	12,284
Total	59,899	59,899

¹<https://github.com/cardiffnlp/tweeteval/tree/main/datasets/sentiment>

Data Description . Before training, we conducted EDA to understand the dataset’s key characteristics, including sentiment label distributions, tweet length, special tokens, and duplicates.

Tweet Length Statistics Length of tweets was calculated by words and characters number in tweets.

Table 2. Average Tweet Length

Dataset	Average Words	Average Characters
Training	19.2	108.0
Validation	19.4	108.6
Testing	14.9	92.4

Training and validation tweets have similar average lengths of approximately 19 words, while the test set is shorter at 14.9 words. This difference should be considered when interpreting model generalization on the test set.

Duplicate Analysis Duplicate tweets were identified by comparing tweet texts within each dataset split.

Table 3. Duplicate Tweets

Dataset	Duplicate Tweets	Percentage
Training	28	0.06%
Validation	0	0.00%
Testing	0	0.00%

Only 28 duplicate tweets (0.06%) were found in the training set, while no duplicates were detected in the validation or testing sets. The duplicate rate is negligible, so no duplicate removal was applied.

Visualization. In order to give a better characterization of the dataset several visualization tools were used.

Tweet Length Distribution We can see that the distribution is such that most tweets are between 15 and 25 words long, so the length of the text is fairly uniform across the dataset. Moreover, the length distributions of the three sentiment classes appear to be almost identical and there are no meaningful differences in the shape of the distributions. This means that the length of tweets alone is not a very good feature to discriminate between the sentiments. Rather, the meaning of the words, the context and the linguistic features of the tweets will have a much bigger influence on which sentiment they express.

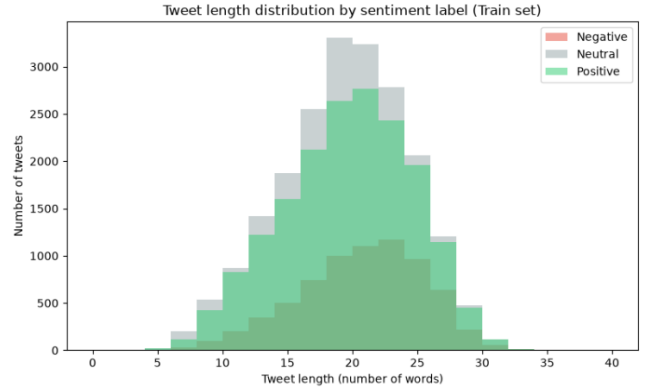


Figure 1. Tweet Length Distribution by Sentiment Label

4. Methodology

4.1. Baseline Model: TF-IDF + LinearSVC

As a fast baseline, we combine TF-IDF features with a Linear Support Vector Classifier (LinearSVC), giving a low-cost comparison point before the transformer-based models (BERT, RoBERTa). The pipeline is shown in Figure 2.

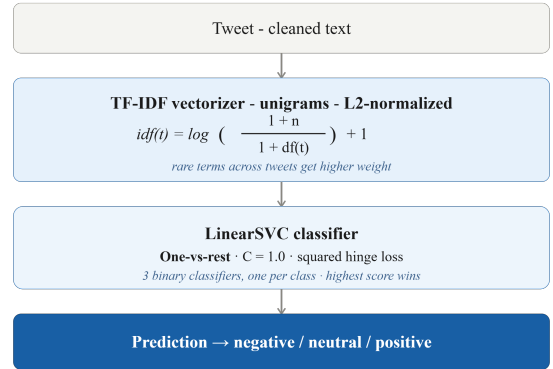


Figure 2. TF-IDF + LinearSVC baseline pipeline.

Feature Extraction. Tweets are vectorized using scikit-learn’s TfidfVectorizer [9] with default settings. The final TF-IDF weight of term t in tweet d is:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t) \quad (1)$$

where $\text{tf}(t, d)$ is the frequency of term t in tweet d , and $\text{idf}(t)$ follows scikit-learn’s smoothed formula shown in Figure 2. In other words, a word is given more weight if it appears often in a given tweet but rarely across the rest of the dataset, making it more useful for distinguishing that tweet’s sentiment.

Model. LinearSVC [10] is chosen for its speed and effectiveness on high-dimensional sparse text data,

trained with a one-vs-rest scheme:

$$f_k(x) = w_k^T x + b_k, \quad \hat{y} = \arg \max_k f_k(x) \quad (2)$$

In other words, each class is scored separately by its own linear function, and the tweet is assigned to whichever class receives the highest score. We use scikit-learn’s default hyperparameters, except for increasing `max_iter` to 5000 to ensure convergence on the high-dimensional TF-IDF features. The vectorizer is fit only on the training set and used to transform the validation and test sets, avoiding data leakage.

4.2. Reproduced Transformer Models: BERT and RoBERTa

We fine-tune two stronger models, BERT [6] and RoBERTa [7]. Both are encoder-only Transformers [5] with $N = 12$ stacked layers and $h = 12$ attention heads, using scaled dot-product self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

so that each token’s reflects its full context (e.g. a preceding negation), unlike the bag-of-words baseline.

Also, they share the same architecture and objective, so any performance difference observed in Section 5 stems from pretraining choices rather than architecture: RoBERTa uses *dynamic masking* (a new mask sampled every time a sequence is seen, rather than fixed once during preprocessing), removes the *Next Sentence Prediction* objective entirely, is trained on substantially more data (~160GB vs. BERT’s 16GB), and uses a byte-level BPE tokenizer with a larger, 50K-token vocabulary instead of BERT’s 30K-token WordPiece vocabulary [7]. Figure 3 summarize the pipeline for the two model.

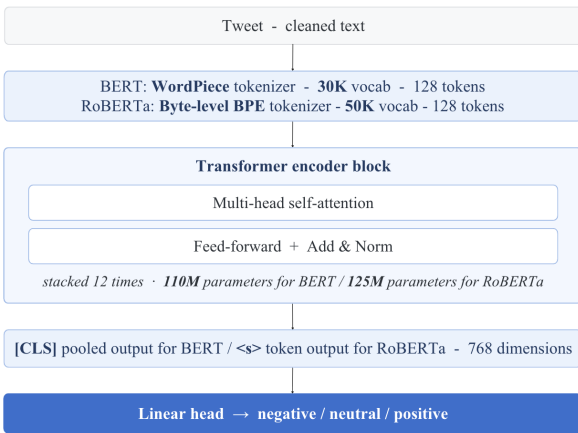


Figure 3. BERT and RoBERTa pipeline

We fine-tune `bert-base-uncased` (110M parameters) and `roberta-base` (125M parameters) end-to-end

using Hugging Face’s `AutoModelForSequenceClassification`, which attaches a linear classification head over the pooled sentence representation and outputs a softmax over the three sentiment classes. Both models were trained under an identical hyperparameter configuration (Table 4) to ensure that any performance gap reflects differences in pretraining rather than differences in the fine-tuning setup.

Table 4. Fine-tuning hyperparameters for BERT and RoBERTa.

Hyperparameter	Value
Max sequence length	128 tokens
Learning rate	2×10^{-5}
LR scheduler	Linear
Batch size (train / eval)	16 / 32
Weight decay	0.01
Number of epochs (max)	3
Warmup ratio	0.1
Early stopping patience	2 evaluation epochs
Checkpoint selection metric	Validation Macro-F1
Random seed	42

Macro-F1 is used as the checkpoint-selection criterion for both models: since the test set exhibits a different label distribution than the training set (Section 3), the checkpoint that minimizes loss does not necessarily correspond to the checkpoint with the best classification performance across all sentiment classes.

4.3. OUR MODEL: FINE-TUNING WITH LORA

We apply Low-Rank Adaptation (LoRA) [4] to `roberta-base` to test whether comparable performance can be reached while **updating far fewer parameters**. LoRA freezes the pretrained backbone and instead learns a pair of small low-rank matrices, A and B , injected into the query and value projections of each self-attention layer, following [4], since these were found to be the most impactful targets. We fine-tune four variants with rank $r \in \{4, 8, 16, 32\}$ under an otherwise identical training setup to the full fine-tuning run, which LoRA typically requires given its much smaller number of trainable parameters.

Figures 4 and 5 contrast the two pipelines. Both share the same tokenizer, Trainer loop, per-epoch validation, and final `load_best_model_at_end` step before predicting on the 12,284-tweet test set — the only thing that changes is **which parameters receive gradients**:

- **Full fine-tuning** (Fig. 4): the entire backbone and classification head are trainable, **all 124,647,939 parameters (100%)** are updated every step.
- **LoRA** (Fig. 5): the backbone weight W_0 is **frozen**; only the injected adapters A, B at

each query/value projection receive gradients, leaving just **0.59%-1.4% of parameters trainable**, depending on r .

Since everything else in the two pipelines is identical, any difference in accuracy or training cost reported in Section 5 is attributable to the fine-tuning strategy alone.

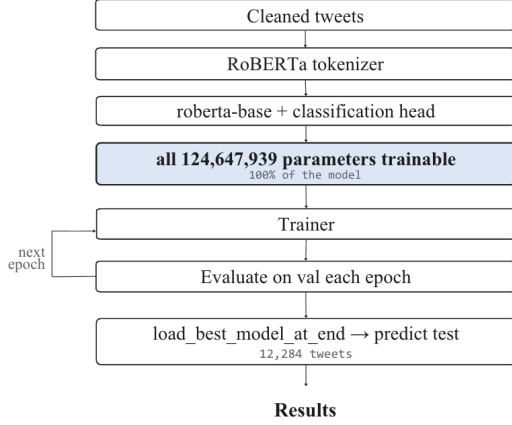


Figure 4. RoBERTa original full fine-tuning pipeline: 100% of parameters trainable.

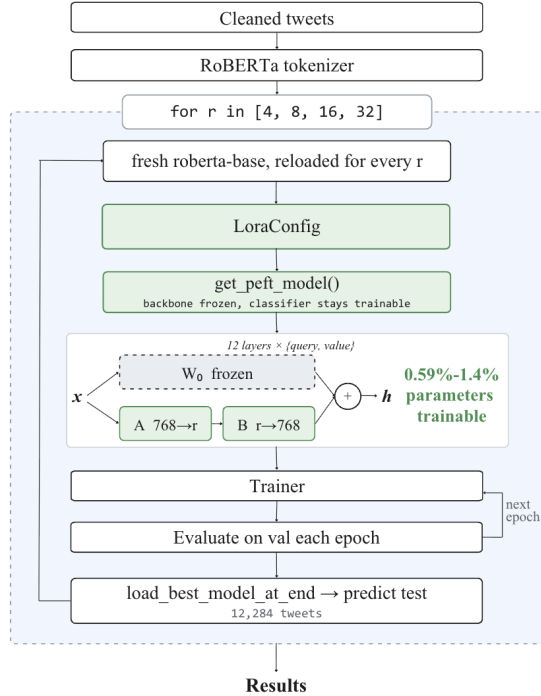


Figure 5. Our LoRA integrated pipeline: backbone frozen, only adapters A, B trainable (0.59%-1.4%).

Parameter count by rank (r). For each self-attention layer, LoRA adds $A \in \mathbb{R}^{768 \times r}$ and $B \in \mathbb{R}^{r \times 768}$ to both the

query and value projections, giving $36,864r$ trainable parameters across all 12 layers. Adding the 593,667 parameters of the (newly initialized, always-trainable) classification head gives the total in Table 5.

r	Trainable params	% of 124.6M
4	740,123	0.59%
8	888,579	0.71%
16	1,183,491	0.94%
32	1,773,315	1.40%

Table 5. Trainable parameters by LoRA rank r .

4.4. OUR ERROR ANALYSIS FRAMEWORK

Accuracy alone cannot separate tweets that are intrinsically hard from tweets that one particular model happens to miss. We therefore designed a three-stage framework that isolates the errors shared by every architecture and attributes each one to a linguistic cause. Figure 6 summarises the full procedure.

Stage 1: shared error set. We first evaluate the three reproduced baselines - TF-IDF + LinearSVC, BERT, and RoBERTa - on the identical held-out test split, then build one error set per model, $E_m = \{i : \hat{y}_i^{(m)} \neq y_i\}$. Aligning the three sets on the pair (tweet text, gold label) and taking their intersection $E_{\text{SVM}} \cap E_{\text{BERT}} \cap E_{\text{RoBERTa}}$ yields the tweets that no architecture classifies correctly. Restricting the analysis to this intersection removes model-specific noise: whatever remains is a property of the task rather than of any single model.

Stage 2: cause attribution. None of these tweets carries a labelled reason for being hard, so each is passed through a rule cascade of eight categories tested in a fixed priority order: sarcasm/irony, mixed sentiment, negation, news/quotation, missing context, slang, very short, and a catch-all for tweets that match nothing. Each test is a keyword list or regular expression applied to the normalised tweet. Sarcasm matches a fixed phrase list ("yeah right", "/s", "just what i needed"); mixed sentiment fires when a positive and a negative lexicon token co-occur, or when a contrast cue (but, however) accompanies a sentiment word; negation uses a regular expression over not, never, no, n't; missing context fires on the presence of @user or a link, or on a leading bare pronoun (this, that, it); very short fires when four or fewer words remain after URLs, mentions, and punctuation are stripped. The cascade is exclusive and order-sensitive, the first rule that fires is kept as the primary cause, so every tweet leaves with exactly one label and the resulting shares sum to 100%.

Stage 3: cross-model agreement. For each category we further compute the share of tweets on which all three models predict the *exact same* wrong label. High

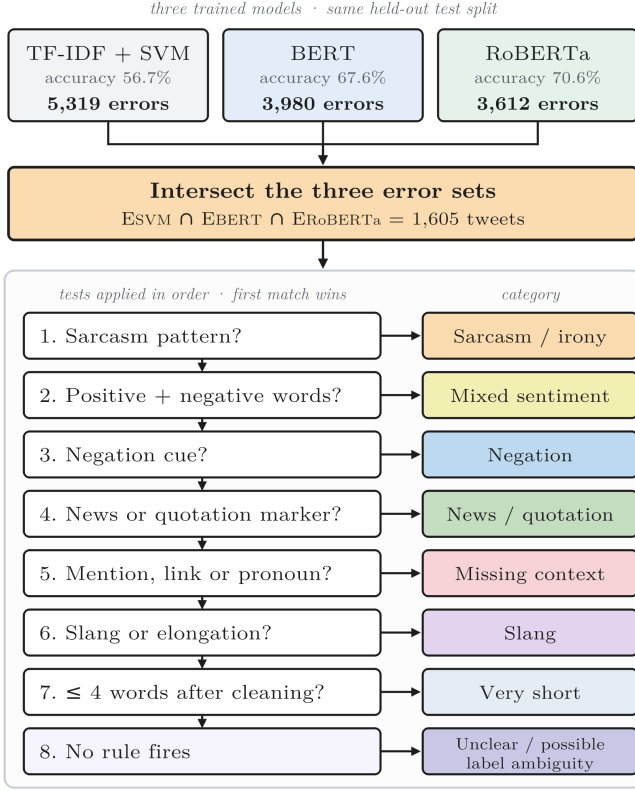


Figure 6. Our error analysis framework: the three baselines are evaluated on the same held-out test split, their error sets are intersected into a shared error set of 1,605 tweets, and every shared error is assigned exactly one cause by an ordered rule cascade in which the first matching rule wins.

agreement indicates a single misleading surface cue or a debatable gold label that steers every architecture identically; low agreement indicates a tweet that is intrinsically ambiguous and that each model resolves differently.

5. Experimental Results

5.1. Reproduced Baselines: LinearSVC, BERT, and RoBERTa

Throughout this section, for a given class c we let TP_c , FP_c , and FN_c denote true positives, false positives, and false negatives:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (4)$$

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (5)$$

Macro-F1 averages $F1_c$ equally across the three classes, while Weighted-F1 weights each class’s F1 by its support:

$$\text{Macro-F1} = \frac{1}{3} \sum_c F1_c \quad (6)$$

$$\text{Weighted-F1} = \sum_c \frac{n_c}{N} F1_c \quad (7)$$

The TF-IDF + LinearSVC baseline reached 56.68% test accuracy (Macro-F1 0.55), fine-tuned BERT reached 67.60% (Macro-F1 0.67), and fine-tuned RoBERTa reached 70.58% (Macro-F1 0.71) - an improvement of 14.4 and 10.3 accuracy points over the baseline, respectively, shown in table 6 below.

Table 6. Test set performance comparison across the three reproduced approaches.

Model	Accuracy	Macro-F1	Weighted-F1
TF-IDF + LinearSVC	0.57	0.55	0.57
Fine-tuned BERT	0.67	0.67	0.67
Fine-tuned RoBERTa	0.71	0.71	0.71

The baseline’s confusion matrix (Figure 7) shows both minority classes are frequently pulled toward Neutral: 41.7% of Negative tweets (1,657/3,972) and 33.9% of Positive tweets (805/2,375) are misclassified as Neutral, consistent with its weakest class being Positive ($F1 = 0.52$), driven by low precision (0.46) - the baseline tends to *over-predict* Positive from strong lexical cues (e.g. “great”, “love”) regardless of context.

BERT and RoBERTa instead *under-recall* Neutral, their weakest class, at 0.6842 and 0.63 recall respectively, confusing genuinely neutral tweets (e.g. factual statements, headlines) with either polarity. For BERT, Negative shows high precision but low recall (0.768 / 0.583): 87.5% of its 1,658 missed Negative tweets are pulled toward Neutral (Figure 8); Positive shows the mirror pattern (recall 0.765, precision 0.584). RoBERTa’s confusion matrix (Figure 9) shows the same dominant pattern: Neutral tweets are its single largest source of error, split between Negative and Positive misclassification. This recall gap persists across both Transformer models and is consistent with the class-distribution shift identified during data description: Positive accounts for only ~19% of the test set versus ~39% in training, so validation-time improvements do not transfer one-to-one to the test distribution.

5.2. Our Model: LoRA Fine-tuning Results

LoRA matches full fine-tuning while updating under 1.5% of parameters and training up to 39% faster. Table 7 compares full fine-tuning against LoRA at four ranks ($r \in \{4, 8, 16, 32\}$), all measured under identical training infrastructure and hyperparameters so that trainable parameters, training time, and accuracy

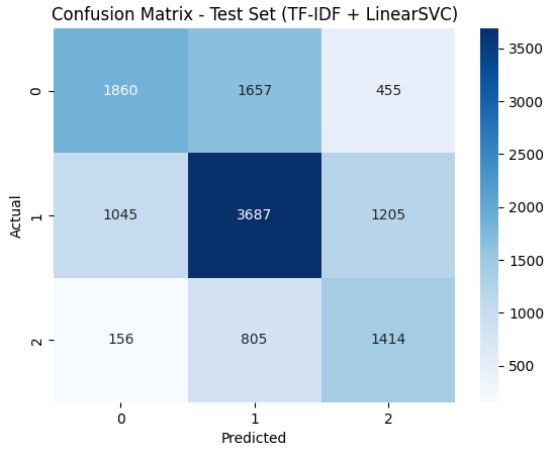


Figure 7. Confusion matrix of the TF-IDF + LinearSVC baseline on the test set

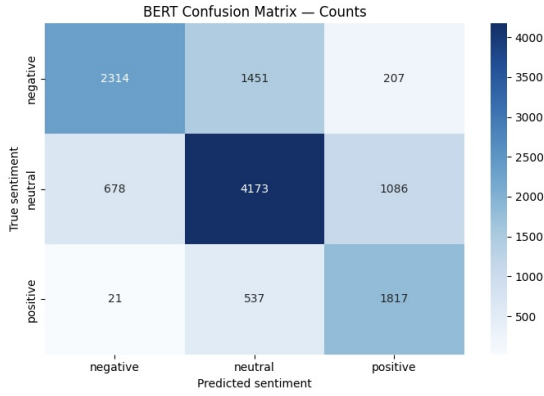


Figure 8. Confusion matrix of fine-tuned BERT

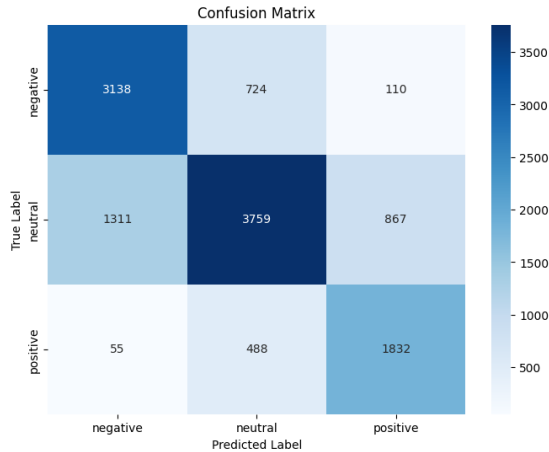


Figure 9. Confusion matrix for RoBERTa on the test set.

are directly comparable. Full fine-tuning updates all 124.6M parameters of RoBERTa and requires 18.8 minutes of training. Every LoRA configuration tested instead updates only 0.59%-1.40% of the

model’s parameters and completes training in 11.5-13.7 minutes, a reduction of 27%-39% in wall-clock training time, while matching or marginally exceeding full fine-tuning on both test accuracy and Macro-F1 at every rank, with the best configuration ($r = 32$) reaching 71.29% accuracy and 0.7129 Macro-F1 versus 70.58% and 0.7069 for full fine-tuning. This indicates that full-parameter updates are not strictly necessary to reach RoBERTa-tweetter’s performance ceiling on TweetEval sentiment: a low-rank adaptation captures nearly all of the task-relevant signal, while leaving the vast majority of the pretrained weights untouched.

Table 7. Full fine-tuning vs. LoRA: trainable parameters, training cost, and test performance.

Method	Params (%)	Time (min)	Acc.	Macro-F1
Full fine-tune	124.6M (100%)	18.8	0.7056	0.7069
LoRA $r = 4$	740K (0.59%)	13.7	0.7106	0.7111
LoRA $r = 8$	888K (0.71%)	11.5	0.7123	0.7124
LoRA $r = 16$	1.18M (0.94%)	11.6	0.7125	0.7128
LoRA $r = 32$	1.77M (1.40%)	11.5	0.7129	0.7129

Peak GPU memory is nearly identical across all configurations (≈ 4 GB), since the frozen base model still dominates memory usage regardless of how few parameters are trainable; the savings from LoRA appear in optimizer state and gradient memory, which are small relative to model weights at this scale and therefore do not show up clearly at $r \leq 32$. When these results are placed alongside the reproduced baselines (Table 8), LoRA matches full RoBERTa fine-tuning on every metric while updating only a fraction of its parameters.

Table 8. Test set performance comparison.

Model	Accuracy	Macro-F1	Weighted-F1
TF-IDF + LinearSVC	0.57	0.55	0.57
Fine-tuned BERT	0.67	0.67	0.67
Fine-tuned RoBERTa	0.71	0.71	0.71
RoBERTa + LoRA ($r = 32$)	0.71	0.71	0.71

Rank increase shows performance improves early. Raising the LoRA rank from $r = 4$ to $r = 32$ yields only a marginal Macro-F1 gain ($0.7111 \rightarrow 0.7129$), despite a $2.4\times$ increase in trainable parameters. This suggests a small rank (e.g. $r = 8$) already captures nearly all of the benefit achievable through low-rank adaptation on this task - a practical consideration for deployment under tighter compute or storage budgets.

5.3. Error Analysis

Applying the framework of Section 4.4 to the three reproduced baselines yields 1,605 shared errors,

13.07% of the 12,284 test tweets, that no architecture classifies correctly. Negative tweets are somewhat overrepresented in this set (35.8% versus 32.3% of the full test set), consistent with the recall gap already observed for both the baseline and BERT.

Table 9. Cause of failure for the 1,605 tweets misclassified by all three models.

Category	Count (%)
Unclear / label ambiguity	674 (42.0%)
Missing context	541 (33.7%)
Negation	227 (14.1%)
News/quotation	80 (5.0%)
Mixed sentiment	41 (2.6%)
Slang	22 (1.4%)
Very short	17 (1.1%)
Sarcasm/irony	3 (0.2%)

Table 9 and Figure 10 report the distribution over the eight categories. Unclear or ambiguous cases and missing context together account for roughly three quarters of every shared failure. Negation is the largest single identifiable language pattern at 14.1%. The remaining five categories cover only about 9% of the shared errors, placing the more frequently discussed failure modes - sarcasm in particular, at 0.2% - in a small minority of what defeats all three architectures at once.

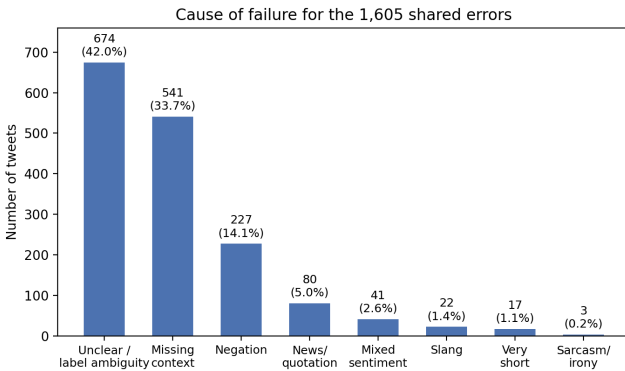


Figure 10. Cause of failure for the 1,605 shared errors, by category.

Across all 1,605 shared errors the three models predict the exact same wrong label 74.8% of the time and disagree the remaining 25.2%. Figure 11 breaks this down by category: mixed sentiment shows the lowest agreement (48.8%), while slang (81.8%) and unclear cases (77.7%) show the highest.

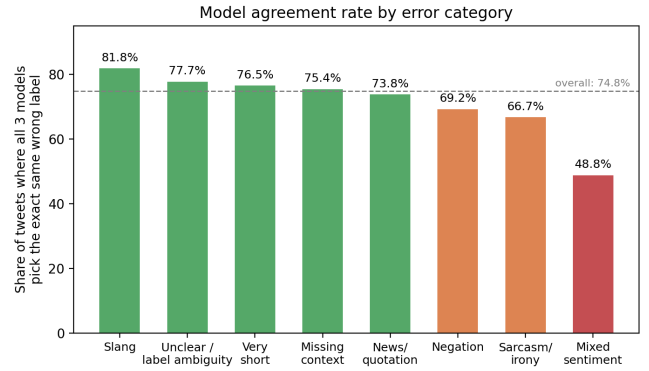


Figure 11. Share of tweets in each category where all three models pick the exact same wrong label.

6. Discussion

Transformer models significantly outperform the baseline. Both Transformer models outperform the TF-IDF + LinearSVC baseline, with RoBERTa improving test accuracy by 14.4 points and BERT by 10.3 points. This is consistent with the SVM’s bag-of-words representation lacking any notion of word order or negation scope, unlike the self-attention used by both Transformer models.

RoBERTa outperforms BERT. Despite nearly identical architecture, RoBERTa outperforms BERT on all metrics, mainly because of its stronger pretraining recipe (larger corpus, dynamic masking, no Next Sentence Prediction). Both models were fine-tuned under identical hyperparameters (Section 4.2) to isolate this effect.

LoRA’s efficiency gain comes at essentially no accuracy cost. The results in Section 5.2 are the clearest actionable finding of this study: full-parameter updates are not required to reach RoBERTa-tweets’ performance ceiling on this benchmark, and the residual gap between LoRA and full fine-tuning is small, localized to negation-heavy cases, and outweighed by the training-cost savings.

All models struggle most with the Neutral class in recall, but for different reasons. BERT and RoBERTa both obtain their lowest recall on Neutral, while the SVM’s weakest class is Positive, driven by low precision rather than low recall. This suggests the baseline over-predicts Positive via strong lexical cues, while the Transformer models instead under-recall Neutral, confusing factual/neutral tweets with either polarity, consistent with the class-distribution shift between train (~39% Positive) and test (~19%).

Shared errors are not driven by sarcasm. A recent meta-review identifies sarcasm as the most commonly cited challenge in sentiment analysis [8], so we tested this directly on the 1,605 tweets (13.07% of the test set) that the baseline, BERT, and RoBERTa all fail on. If sarcasm were the real bottleneck, it should dominate this intersection; instead it accounts for only 0.2% of shared errors, while unclear labels (42.0%) and missing context (33.7%) explain three quarters of them. This suggests self-attention already handles sarcasm reasonably well, and that the field’s emphasis on it may reflect difficulty for older lexicon-based methods more than for Transformer models – the real bottleneck here is the data itself, not sarcasm-specific tooling.

Summary. The overall ranking is as follow: RoBERTa $\approx$ RoBERTa+LoRA > BERT > TF-IDF+LinearSVC (considering accuracy, Macro-F1, and Weighted-F1). This performance can be retained with LoRA updating as little as 0.59% of parameters and training up to 39% faster. The error analysis in Section 5.3 further shows that none of the four approaches fully resolves Neutral-class confusion, and that the dominant sources of failure across all architectures is label ambiguity and missing context, which are structural properties of the task itself rather than weaknesses specific to any one model.

7. Conclusion

This study compared four approaches to three-way sentiment classification on the TweetEval benchmark. The lexical TF-IDF + LinearSVC baseline reached 56.68% test accuracy, fine-tuned BERT 67.60%, and fine-tuned RoBERTa 70.58%, confirming that self-attention over contextual embeddings substantially outperforms a bag-of-words representation on short, informal text.

Our first contribution goes beyond this reproduction: applying Low-Rank Adaptation to roberta-base and sweeping the rank over $r \in \{4, 8, 16, 32\}$, we found that LoRA matches or marginally exceeds full fine-tuning at every rank while updating only 0.59%-1.40% of the model’s parameters and cutting wall-clock training time by 27%-39%. The best configuration ($r = 32$) reached 71.29% accuracy and 0.7129 Macro-F1, against 70.58% and 0.7069 for full fine-tuning. Performance also saturates early: raising the rank from $r = 4$ to $r = 32$ bought only +0.18 Macro-F1 points despite a $2.4\times$ increase in trainable parameters, so a small rank already captures essentially all of the available benefit. Full-parameter updates are therefore not required to reach this benchmark’s performance ceiling.

Our second contribution is a cross-model error analysis that asks which tweets defeat every architecture at once. Intersecting the error sets of the three baselines

isolated 1,605 tweets (13.07% of the test set) that none of them classifies correctly, and attributing a cause to each through an ordered rule cascade produced a distribution that runs against the conventional account of Twitter sentiment: label ambiguity (42.0%) and missing context (33.7%) together explain roughly three quarters of all shared failures, while sarcasm (cited as the central difficulty of this task) accounts for 0.2%. That the three models converge on the exact same wrong label 74.8% of the time indicates these errors are driven by properties of the data rather than by any model’s bias: a tweet whose sentiment lives in a linked article, or whose gold label is genuinely debatable, is unreachable by architecture or optimisation alone. Two limitations qualify this finding. The cause attribution is a heuristic over keyword lists and regular expressions rather than verified annotation, and its first-match-wins design credits a multi-cause tweet to only one category, so the counts should be read as an approximate map. Future work should therefore manually annotate a sample of the catch-all category to separate genuine label noise from patterns the cascade does not test for, evaluate Twitter-specific pretrained encoders, and extend LoRA beyond the query and value projections to test whether a wider adapter placement narrows the small residual gap on negation-heavy cases.

8. Member contribution

Table 10. Division of work and contribution of each group member.

Member	Tasks	%
Ngo Thi Kim Ngan	Fine-tunes RoBERTa and integrates LoRA; compares models’ results; writes the Introduction and Discussion sections of the report.	25%
Tong Khanh Ly	Fine-tunes BERT; conducts the error analysis; writes the Conclusion.	25%
Nguyen Quang Minh	Builds the two preprocessing pipelines and trains the TF-IDF + LinearSVC baseline model.	25%
Nguyen Ngoc Anh	Loads the dataset and performs exploratory data analysis; writes the Literature Review.	25%
Total		100%

References

- [1] B. Pang and L. Lee, “Opinion mining and sentiment analysis,” *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, 2008.

-
- [2] F. Barbieri, J. Camacho-Collados, L. Neves, and L. Espinosa-Anke, "Tweeteval: Unified benchmark and comparative evaluation for tweet classification," 2020. [Online]. Available: <https://arxiv.org/abs/2010.12421>
- [3] S. Rosenthal, N. Farra, and P. Nakov, "SemEval-2017 task 4: Sentiment analysis in Twitter," in *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, S. Bethard, M. Carpuat, M. Apidianaki, S. M. Mohammad, D. Cer, and D. Jurgens, Eds. Vancouver, Canada: Association for Computational Linguistics, Aug. 2017, pp. 502–518. [Online]. Available: <https://aclanthology.org/S17-2088/>
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [7] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>
- [8] L. E. M. Ayravainen, J. Hinds, and B. I. Davidson, "Sentiment analysis in digital spaces: An overview of reviews," 2023. [Online]. Available: <https://arxiv.org/abs/2310.19687>
- [9] Pedregosa, F. et al., "Tfidfvectorizer — scikit-learn documentation," https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html, accessed: 2026-08-05.
- [10] —, "Linearsvc — scikit-learn documentation," <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>, accessed: 2026-08-05.